

FORTNIGHTLY PROJECT REPORT

Content Personalisation Agent

A full-stack multi-agent AI platform delivering per-need personalised content across 7 domains and 34 specialised agents.

REPORT DATE

September 27, 2026

REPORT TYPE

Fortnightly Technical

STATUS

✓ All Tests Passing

34

AI Agents

7

Domains

163

Tests Passing

3

Profile Levels

Table of Contents

01 Project Overview

02 What Problem Does It Solve?

03 How It Works — The Big Picture

04 The 7 Domains and 34 Agents

05 Key Features & Functionality

06 Tech Stack

07 System Architecture

08 The Personalisation Engine

09 AI & Model Support

10 Security

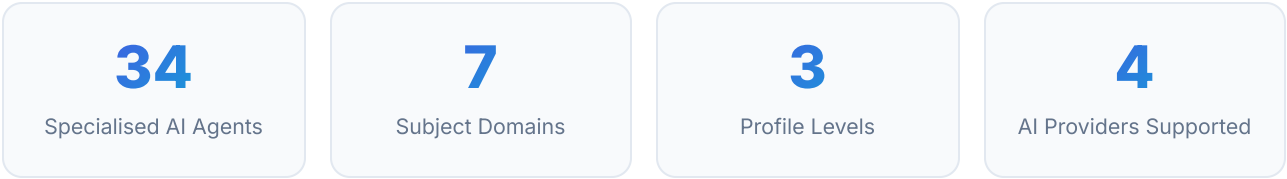
11 Project Structure

12 Current Status & Verified Results

1 Project Overview

The **Content Personalisation Agent** is a full-stack, AI-powered web application that provides deeply personalised content and assistance across 7 different subject areas (called "domains"). Instead of one general-purpose chatbot, the system uses **34 narrow, specialised AI agents** — each designed to do exactly one job extremely well.

 **Core Idea:** Personalisation is *per need*, not *per subject*. If you're a Class 10 student studying maths, you get a *Class 10 Maths Tutor* — not a generic education assistant. If you need a LinkedIn post image, you get the *Social Post Image Generator* which returns an actual PNG file — not a paragraph of text.



Think of it like having a team of 34 dedicated experts, where each expert knows only their own job — and if you ask the wrong expert, they politely direct you to the correct one instantly, without wasting any AI computation.

2 What Problem Does It Solve?

Most AI chatbots are "one-size-fits-all" — they try to do everything but personalise nothing. This project targets three specific problems:

Problem	How This Project Solves It
Generic, impersonal responses	Every reply is customised using 3 stacked layers of user profile — global preferences, domain-specific settings, and agent-specific configuration
No memory of user preferences	A dedicated memory system stores user preferences, corrections, and recurring needs — and retrieves them semantically for each session
AI scope creep	Each agent has a strict scope — out-of-scope requests trigger an instant, deterministic redirect to the correct agent with zero model cost

Here is the simplified journey of what happens when a user interacts with the system:

1

Login / Sign Up

User authenticates via JWT-secured registration or login

2

Browse Domains & Agents

Dashboard shows 7 domain cards → click to explore the agents inside each

3

Two-Step Agent Setup

First use prompts a guided setup: Step 1 fills domain-shared info, Step 2 fills agent-specific info

4

Send a Message

User types a request. The 10-step LangGraph pipeline kicks in behind the scenes

5

Personalised Response

A text answer, a branded PNG image, or a data chart is returned — streamed in real-time

6

Feedback & Learning

User rates the response; corrections become permanent memories and profile updates

The 7 Domains and 34 Agents

The system organises its 34 agents into 7 subject-specific domains. Each domain has a **shared profile** that all its agents inherit, plus each agent has its own specific profile.



Education

6 agents · board, language, exam target

Class 10 Maths Tutor · Class 10 Science · Class 12 Physics · Homework Solver · Exam Planner · Quiz Generator



Technical

6 agents · experience, style, project context

Python Engineer · React Developer · SQL Expert · DSA Coach · Debugger · Code Reviewer



Career

4 agents · role, skills, location

Resume Writer · Interview Coach · Job Search Advisor · Skill Roadmap Planner



Marketing

5 agents · brand, audience, colours

Social Post Image Generator  PNG
Ad Creative Generator  PNG
Social Caption Writer · SEO Optimiser · Campaign Planner



Analytics

4 agents · tools, data context

Chart Builder  Chart
EDA Planner · SQL Analytics Expert · Insight Writer



Research

4 agents · field, depth, background

Concept Explainer · Summariser · Literature Brief · Source Finder



Creative

5 agents · voice, tone, influences

Story Writer · Script Writer · Idea Generator · Copywriter
Poster Image Generator  PNG

Note on output types: 30 agents return **text**. 3 agents return **PNG image files** (social post, ad creative, poster). 1 agent returns a **rendered chart**. The `GET /api/catalog` endpoint reports these counts live.

5.1 Three-Level Personalisation

Every AI response is built using three stacked profile layers, assembled most-general to most-specific:

Level	What It Contains	Example
Global Profile	Name, occupation, skill level, preferred response style, language	"Intermediate level, concise responses"
Domain Profile	Shared across all agents in the domain — set once, used everywhere	"Brand: Acme Cloud, Tone: Professional, Audience: Developers"
Agent Profile	Specific to exactly one agent's job	"Platform: LinkedIn, Style: Bold typography, Logo: ACME CLOUD"

5.2 Smart Memory System

The system builds durable long-term memory of user preferences:

- **Selective capture** — Only meaningful signals are stored (preferences, corrections, requirements), not raw conversation history
- **Reinforcement** — A near-duplicate memory increments a counter and raises importance instead of creating a duplicate row
- **Three scopes** — Agent-scoped (only that agent), domain-scoped (all agents in the domain), or global (every agent)
- **Hybrid retrieval** — Semantic search (vector embeddings) plus "standing" pinned preferences that are always included regardless of topic

5.3 Scope Guard — Agents Stay in Their Lane

```
# Example scope guard in action You → Class 10 Maths Tutor: "Explain Python decorators" → Instant redirect: "That is outside what I do. I am the Class 10 Maths Tutor. Python Engineer handles this." [ 0.0s, no model call ]
```

This happens **before any AI model call** — a deterministic keyword scoring system detects the mismatch and redirects instantly. The UI shows an amber "Switch to [Agent]" button.

5.4 Router Agent

When the user doesn't explicitly pick an agent, the Router automatically finds the right one using a priority chain:

Priority	Method	Speed
1st	Explicit user selection always wins	Instant
2nd	Keyword/phrase scoring across all 34 agents — if one has a clear margin, done	Instant
3rd	Two-stage LLM classification: pick domain first (7 options), then agent in domain (4–6 options)	Fast (2 small decisions)
4th	Best keyword match, then existing conversation agent, then Concept Explainer as default	Instant

5.5 Image Generation (No GPU Required)

Four agents produce actual **PNG image files** instead of text. The approach: the AI generates a JSON specification (headline, colours, layout, CTA), and a local Pillow-based card renderer produces the image. This means:

- Works on any machine — no GPU, no API key needed by default
- Brand colours from the domain profile are automatically applied
- Optional upgrade to `gpt-image-1` for photographic output
- Images are served with download links, alt text, and reload correctly from history

5.6 Chart Generation (Real Data Only)

The Chart Builder uses matplotlib to plot *your actual numbers*. It explicitly refuses to invent data — if you ask for "a chart of something interesting" with no data, it asks you to provide numbers. Given real data, it renders a 1200×700 PNG chart.

5.7 Real-Time Streaming

Because local AI models can be slow (5 tokens/second on a CPU-only machine), the system uses **Server-Sent Events (SSE)** to stream text token by token. The UI shows a live elapsed timer, Stop button, and the right panel pre-loads profile/memory context while the model is still generating. Falls back to standard POST if a proxy blocks SSE.

5.8 Feedback Loop

User Action	System Response
👍 Thumbs up	Positive signal recorded
👎 Thumbs down (no text)	Recorded, no automatic changes
👎 + explanation text	Stored as memory immediately (next request benefits)
Same correction twice	Permanently updates the agent/global profile

6

Tech Stack

FRONTEND

- ▶ **React 18** UI framework
- ▶ **Vite** Build tool & dev server
- ▶ **Tailwind CSS** Styling
- ▶ **React Router** Page navigation
- ▶ **Axios** API communication
- ▶ **react-markdown** Render AI output
- ▶ **Lucide Icons** Icon set

BACKEND

- ▶ **Python 3.12+** Language
- ▶ **FastAPI** Web API framework
- ▶ **LangGraph** Agent orchestration
- ▶ **SQLAlchemy 2** Database ORM
- ▶ **Alembic** DB migrations
- ▶ **Pydantic v2** Data validation
- ▶ **PyJWT + bcrypt** Auth & security

DATABASE & AI

- ▶ **PostgreSQL 16** Primary database
- ▶ **pgvector** Vector similarity search
- ▶ **SQLite** Dev/offline fallback
- ▶ **Ollama** Local AI models (free)
- ▶ **OpenAI API** Hosted AI provider
- ▶ **AWS Bedrock** Enterprise AI
- ▶ **Pillow + Matplotlib** Image & chart rendering

Layer	Technology	Purpose
Containerisation	Docker + Docker Compose	One-command full-stack deployment
Agent Orchestration	LangGraph	Compiles and runs the 10-node chat pipeline as a directed graph
Vector Search	pgvector (PostgreSQL extension)	Stores and semantically searches memory embeddings
Streaming	Server-Sent Events (SSE)	Delivers AI tokens to the browser in real-time

The LangGraph Pipeline (10 Steps)

Every chat message passes through this exact 10-node pipeline compiled by LangGraph:

```
Step 1: load_user → Identify who is sending the message
Step 2: router → Decide which agent handles this request
Step 3: scope_guard → Check if the request is within the agent's job
Step 4: load_global → Load the user's global preferences
Step 5: load_domain → Load the domain-level profile (e.g. brand info)
Step 6: load_agent → Load the agent-specific profile
Step 7: retrieve_memory → Semantic + standing memory retrieval
Step 8: build_context → Assemble all layers into the AI prompt
Step 9: generate → Call AI → text | image renderer | chart renderer
Step 10: store_message → Save conversation, extract new memories
```

Database Schema

```
users —1:1— global_profiles |—1:N— domain_profiles (unique on user_id + domain; JSONB data)
|—1:N— agent_profiles (unique on user_id + agent_key; JSONB data) |—1:N—
conversations —1:N— messages —1:N— feedback |—1:N— memories (embedding vector; scope: agent > domain > global)
|—1:N— feedback
```

Frontend Route Structure

URL	Page	Purpose
/	Auth	Login / Sign Up
/dashboard	Dashboard	All 7 domain cards with agent counts
/domain/:domain	Domain Page	Agents inside a selected domain
/agent/:agentKey	Chat	Conversation with a specific agent
/agent/:agentKey/setup	Setup	Two-step personalisation setup
/profile	Profile	View/edit all three profile levels
/settings	Settings	Memory management and preferences

This is the heart of the project. For every message, the system assembles context in order from most general to most specific:

Assembly order (most general → most specific):

Global Profile → Domain Profile → Agent Profile → Standing Memories → Semantic Memories → Short-Term History → Current Message

Example for a Marketing request ("Create a LinkedIn post about cutting cloud costs"):

Layer	Content Applied
Global	"Intermediate skill level, prefers concise responses"
Domain (Marketing)	"Brand: Acme Cloud, Industry: SaaS, Audience: Developers, Tone: Professional, Colours: #4F46E5 #0F172A"
Agent (Post Image)	"Platform: LinkedIn, Style: Bold typography, Logo text: ACME CLOUD"
Standing Memory	"User prefers posts under 150 characters" (pinned — always included)
Semantic Memory	"User previously liked posts with statistics in the headline" (retrieved by similarity)
Message	"Create a LinkedIn post image about cutting cloud costs with automation"

The UI's **right panel** shows exactly which profile data and memories were used — full transparency into how personalisation is working for each response.

9

AI & Model Support

The project supports four interchangeable AI providers — switched with a single environment variable (`LLM_PROVIDER`):

Provider	What It Uses	Cost	Best For
Ollama (recommended)	Local models on your machine (e.g. llama3.2:3b)	Free, no key	Privacy, offline use
OpenAI-compatible	OpenAI, Groq, Together AI, Mistral, OpenRouter...	Paid / free tier	Speed, quality
AWS Bedrock	Claude 3.5 Sonnet, Titan embeddings	AWS pricing	Enterprise, compliance
Mock Provider	Deterministic local composer, no model	Free	Development, testing

Recommended Local Setup (Ollama)

Model	Size	Speed (CPU)	Notes
<code>llama3.2:3b</code>	2.0 GB	~5 tok/s	Good default on CPU
<code>llama3.2:1b</code>	1.3 GB	Fastest	Weaker reasoning
<code>qwen2.5:3b</code>	1.9 GB	~5 tok/s	Best at JSON/structure
<code>llama3.1:8b</code>	~5 GB	Slow (needs GPU)	Noticeably better quality

Embedding model used for semantic memory search: `nomic-embed-text` (274 MB, 768-dim vectors).

bcrypt Password Hashing

Passwords stored as bcrypt hashes — never in plain text. Hashes never leave the database.

JWT Authentication

Bearer tokens with expiry, signature, and type claims verified on every request.

Tenant Isolation

Every database query is scoped by `user_id` — another account's data returns 404, not 403.

Input Validation

Pydantic v2 validates all inputs. Profile data is whitelisted against declared field keys.

SQL Injection Prevention

All queries use SQLAlchemy ORM. Raw vector search query is fully parameterised.

CORS Restrictions

Restricted to configured origins with explicit method/header allowlist.

No Secrets in Browser

No AWS credentials, model keys, or API tokens are ever sent to the frontend.

File Upload Safety

Size (≤ 1 MB), extension whitelist, and content decodability are all validated.

Frontend dependency audit result: 0 vulnerabilities

Folder / File	Purpose
<code>backend/app/agents/definitions/</code>	All 34 agent specs declared as data (7 domain modules)
<code>backend/app/agents/router.py</code>	Keyword + LLM-based agent routing
<code>backend/app/agents/scope.py</code>	Deterministic scope guard + handoff
<code>backend/app/agents/runtime.py</code>	3 generic runtimes: TextAgent, ImageAgent, ChartAgent
<code>backend/app/agents/graph.py</code>	10-node LangGraph chat pipeline
<code>backend/app/llm/</code>	AI provider implementations (Ollama, OpenAI, Bedrock, Mock)
<code>backend/app/memory/</code>	Memory extraction, storage, and vector retrieval
<code>backend/app/media/</code>	Pillow card renderer and matplotlib chart renderer
<code>backend/app/services/personalization.py</code>	Assembles the 3-level context for every request
<code>backend/alembic/</code>	Database migration history (2 migrations)
<code>backend/scripts/</code>	Utilities: seed data, pipeline verification, migration tests
<code>frontend/src/pages/</code>	Auth, Dashboard, Domain, AgentSetup, Chat, Profile, Settings
<code>frontend/src/components/</code>	Sidebar, ChatMessage, PersonalizationPanel, Forms
<code>infra/init-pgvector.sql</code>	Enables pgvector extension on PostgreSQL first-run
<code>docker-compose.yml</code>	Full stack in one command (Postgres + Backend + Frontend)

All tests run on Intel i5-10210U, 4 cores, **no GPU**. Python 3.14.4, Node 20.20.2.

Test / Check	Result
Full pipeline verification (163 assertions: catalog, chat, image, chart, router, feedback, memory, streaming, tenant isolation)	✓ 163/163 passed
All 34 agents producing their declared output type	✓ 34/34 (30 text, 3 image, 1 chart)
Router accuracy across all 34 agents	✓ 21/21 routing cases correct (keyword scoring, no model call)
Live server test with real Ollama model (llama3.2:3b)	✓ All steps passed
Image generation — Social Post (1200×627 PNG, brand colours)	✓ 43.9s, 60,439 bytes, brand colours applied
Chart generation — 5 data points, refuses to fabricate	✓ 53.5s, exact numbers plotted
Scope guard — Python question sent to Maths Tutor	✓ 0.0s redirect, provider=scope-guard
Memory scope isolation — agent memories don't leak	✓ 0 cross-agent results
Database migration (v1 → v2 → v1, with legacy data)	✓ 45/45 assertions, no data loss
Frontend build — all JSX components imported	✓ 26 files, 0 vulnerabilities

Available Services & Endpoints

Service	URL
Frontend Web App	http://localhost:5173
Backend REST API	http://localhost:8000
API Documentation (Swagger UI)	http://localhost:8000/docs
Health Check	http://localhost:8000/health

✓ Project Summary

The Content Personalisation Agent is a production-ready multi-agent AI platform that organises 34 specialised AI agents across 7 domains, personalises every response using a 3-level profile system, maintains semantic long-term user memory, enforces strict agent boundaries, produces real text/image/chart outputs, and runs on any hardware from a CPU-only laptop to enterprise cloud AI. It is fully containerised, thoroughly tested with 163 passing assertions, and implements modern security best practices throughout.